

Doctor Dashboard - Detailed Technical Requirements

Overview

The Doctor Dashboard is a sophisticated AI-powered clinical encounter system that provides real-time assistance during patient visits. It features intelligent pre-visit analysis, live AI scribe functionality, multi-agent coordination, and seamless EMR integration with EMR as the single source of truth.

1. Dashboard Architecture & Layout

1.1 Main Dashboard Interface

Visual Requirements:

- **Page Title:** "Doctor Dashboard" (24px, bold)
- **Subtitle:** "Review your patient schedule and sign off on lab results." (14px, gray)
- **Layout:** Three main sections stacked vertically
- **Refresh Indicator:** Real-time sync status with EMR
- **Background Processing:** Visual indicator when pre-analysis is running

Header Elements:

```
typescript

interface DoctorDashboardHeader {
  title: "Doctor Dashboard";
  lastEMRSync: string; // "Last synced: 2 minutes ago"
  syncStatus: 'synced' | 'syncing' | 'error' | 'offline';
  pendingActions: number; // Count of items needing attention
  currentTime: string; // Real-time clock
}
```

1.2 Dashboard Sections

Section 1: Ready for You Queue

```
typescript
```

```
interface ReadyForYouSection {
  title: "Ready for You";
  subtitle: "These patients have been roomed by the MA and are ready for your visit.";
  patients: ReadyPatient[];
  emptyState: "No patients are currently ready.";
  refreshInterval: 30; // seconds
}

interface ReadyPatient {
  id: string;
  name: string;
  roomNumber: string;
  readyTime: string; // When MA marked as ready
  waitTime: number; // Minutes waiting
  urgencyLevel: 'routine' | 'urgent' | 'stat';
  maHandoffNotes?: string;
  preAnalysisStatus: 'completed' | 'processing' | 'failed';
  hasAlerts: boolean; // Critical labs, allergies, etc.
}
```

Section 2: Today's Full Schedule

typescript

```
interface TodaysScheduleSection {
  title: "Today's Full Schedule";
  subtitle: "All patients scheduled for today, including their current status in the clinic workflow.";
  patients: ScheduledPatient[];
  filterOptions: ['all', 'scheduled', 'checked_in', 'roomed', 'completed'];
  sortOptions: ['time', 'status', 'name'];
}

interface ScheduledPatient {
  id: string;
  name: string;
  appointmentTime: string;
  visitReason: string;
  status: 'scheduled' | 'checked_in' | 'roomed' | 'ready_for_doctor' | 'in_progress' | 'completed' | 'no_show';
  preAnalysisReady: boolean;
  roomNumber?: string;
  estimatedDuration: number; // minutes
  visitType: 'new_patient' | 'follow_up' | 'urgent' | 'procedure';
}
```

Section 3: Lab Results for Final Review

typescript

```
interface LabResultsSection {
  title: "Lab Results for Final Review";
  subtitle: "Review and approve these results to release them to the patient and EMR.";
  results: PendingLabResult[];
  filterOptions: ['all', 'normal', 'abnormal', 'critical'];
  batchActions: ['approve_all_normal', 'flag_for_followup'];
}

interface PendingLabResult {
  id: string;
  patientName: string;
  testName: string;
  testDate: string;
  status: 'normal' | 'abnormal' | 'critical';
  aiInterpretation?: string;
  urgencyFlag: boolean;
  requiresFollowup: boolean;
  technicianNotes?: string;
}
```

2. Pre-Visit Intelligence System (7-14 Day Look-ahead)

2.1 Data Pipeline Architecture

Data Sources & Processing:

typescript

```
interface PreVisitDataPipeline {
  dataSources: {
    emr: "Patient demographics, medical history, medications, allergies";
    labSystems: "Historical lab results, pending orders, reference ranges";
    insuranceAPIs: "Coverage verification, prior authorizations, copays";
    schedulingSystems: "Appointment details, visit types, provider preferences";
  };

  processingSchedule: {
    type: "rolling";
    frequency: "every_4_hours";
    lookAheadDays: 7; // Configurable: 7 or 14 days
    offPeakPriority: true; // Heavy processing during off-hours
  };

  singleSourceOfTruth: "EMR"; // All conflicts resolved in favor of EMR data
}
```

Data Processing Workflow:

typescript

```
interface PreAnalysisWorkflow {  
  step1_dataGathering: {  
    action: "Pull patient data from all sources";  
    timing: "4 hours before appointment";  
    fallback: "Use cached data if real-time unavailable";  
  };  
  
  step2_aiAnalysis: {  
    action: "Generate clinical summary and suggested checklist";  
    timing: "2 hours before appointment";  
    aiModels: ["clinical_summary", "risk_assessment", "care_gap_analysis"];  
  };  
  
  step3_cachePreparation: {  
    action: "Pre-fetch test results and references for voice commands";  
    timing: "1 hour before appointment";  
    includes: ["lab_trends", "imaging_reports", "consultation_notes"];  
  };  
  
  cancellationHandling: {  
    reschedule: "preserve_analysis_data";  
    cancellation: "purge_immediately";  
    noShow: "preserve_for_24_hours";  
  };  
}
```

2.2 Pre-Analysis AI Generation

Clinical Analysis Components:

typescript

```

interface PreVisitAnalysis {
  patientSummary: {
    demographics: PatientDemographics;
    activeConditions: MedicalCondition[];
    medications: Medication[];
    allergies: Allergy[];
    recentVisits: VisitSummary[];
  };

  riskAssessment: {
    chronicDiseaseManagement: RiskScore;
    drugInteractions: InteractionAlert[];
    preventiveCareGaps: CareGap[];
    hospitalReadmissionRisk: number; // 0-100
  };

  suggestedChecklist: {
    vitalSigns: string[];
    labReviews: string[];
    medicationReviews: string[];
    counselingPoints: string[];
    followUpNeeds: string[];
  };

  clinicalAlerts: {
    criticalValues: LabAlert[];
    drugAllergies: AllergyAlert[];
    contraindications: ContraindicationAlert[];
    insuranceLimitations: CoverageAlert[];
  };
}

```

3. Patient Chart Interface & Clinical Encounter

3.1 Chart Navigation & Layout

Patient Chart Header:

```
typescript
```

```

interface PatientChartHeader {
  patientInfo: {
    name: string;
    age: number;
    dateOfBirth: string;
    mrn: string;
    roomNumber?: string;
  };

  visitContext: {
    appointmentTime: string;
    visitReason: string;
    visitType: 'new_patient' | 'follow_up' | 'urgent' | 'procedure';
    duration: number; // Scheduled minutes
  };

  quickActions: {
    requestMAAssistance: boolean;
    emergencyMode: boolean;
    endEncounter: boolean;
  };

  syncStatus: {
    emrConnection: 'connected' | 'syncing' | 'offline';
    lastSync: string;
    pendingChanges: number;
  };
}

```

Chart Layout Sections:

1. **AI-Powered Analysis & Suggested Plan** (Left top)
2. **AI Scribe & Clinical Encounter** (Left bottom)
3. **Today's Vitals** (Right top)
4. **Medication Review** (Right middle)
5. **Past Visits & Vitals** (Right bottom)

3.2 AI-Powered Analysis & Suggested Plan

Pre-Analysis Display:


```

interface AIAnalysisDisplay {
  analysisStatus: 'completed' | 'processing' | 'failed' | 'outdated';

  patientSummary: {
    primaryConditions: string[];
    keyFindings: string;
    riskFactors: string[];
    lastVisitSummary: string;
  };

  suggestedChecklist: {
    item: string;
    completed: boolean;
    priority: 'high' | 'medium' | 'low';
    reasoning?: string;
  }[];

  clinicalAlerts: {
    type: 'allergy' | 'drug_interaction' | 'critical_value' | 'insurance';
    message: string;
    severity: 'info' | 'warning' | 'critical';
    actionRequired: boolean;
  }[];
}

```

Implementation Rules:

1. **Analysis Refresh:** Re-run analysis if patient data changed since last generation
2. **Checklist Interaction:** Items can be checked off during encounter
3. **Alert Handling:** Critical alerts must be acknowledged before proceeding
4. **Mobile Responsiveness:** Collapsible sections for smaller screens

4. Live AI Scribe System

4.1 Real-time Audio Processing

Bandwidth & Connection Management:

typescript

```
interface AudioProcessingConfig {
  primaryMode: 'live_scribe';
  fallbackMode: 'record_and_transcribe';

  bandwidthThresholds: {
    minimumMbps: 2;
    optimalMbps: 5;
    latencyThreshold: 500; // milliseconds
    packetLossThreshold: 5; // percentage
  };

  connectionMonitoring: {
    checkInterval: 10; // seconds
    notificationMethod: 'persistent_banner';
    fallbackTrigger: 'automatic';
    recoveryDetection: 'post_encounter'; // Switch back after patient visit
  };

  offlineResilience: {
    cacheStrategy: 'local_storage_encrypted';
    syncTrigger: 'connectivity_restored';
    conflictResolution: 'emr_wins';
  };
}
```

Live Scribe Interface:

typescript

```

interface LiveScribeInterface {
  audioVisualization: {
    waveform: boolean;
    speakerDetection: boolean; // Doctor vs Patient identification
    confidenceIndicator: boolean;
  };

  transcriptionDisplay: {
    realTimeText: string;
    speakerLabels: ['Doctor', 'Patient', 'Other'];
    confidenceScoring: number; // 0-100 per phrase
    medicalTermHighlighting: boolean;
  };

  controls: {
    startStop: boolean;
    pause: boolean;
    markImportant: boolean; // Flag key moments
    privateNote: boolean; // Off-record annotations
  };

  splitScreenMode: {
    enabled: boolean;
    activationMethods: ['button_click', 'voice_command'];
    voiceCommands: string[]; // "Show latest CBC", "Display A1C trend"
    displayPriority: 'semantic_match_score';
    doctorSelection: boolean; // Choose from multiple matches
  };
}

```

4.2 Multi-AI Agent Coordination

AI Agent Architecture:

```

typescript

```

```
interface AIAgentSystem {
  agents: {
    clinicalScribe: {
      purpose: "Real-time SOAP note generation";
      priority: 1;
      processingTime: "<3_seconds";
    };

    billingCoder: {
      purpose: "Live billing code suggestions";
      priority: 2;
      processingTime: "<5_seconds";
    };

    orderManager: {
      purpose: "Suggested orders and prescriptions";
      priority: 3;
      processingTime: "<5_seconds";
    };

    medicationReconciler: {
      purpose: "Real-time medication review";
      priority: 4;
      processingTime: "<5_seconds";
    };

    clinicalDecisionSupport: {
      purpose: "Drug interactions, contraindications";
      priority: 1; // High priority for safety
      processingTime: "<2_seconds";
    };
  };

  coordinationLogic: {
    conflictResolution: 'doctor_choice';
    failureHandling: 'continue_other_agents';
    resourceManagement: 'gcp_auto_scaling';
    qualityAssurance: 'doctor_review_required';
  };
}
```

Agent Processing Pipeline:

typescript

```
interface AgentProcessingPipeline {  
  inputProcessing: {  
    audioStream: 'real_time_chunks';  
    transcription: 'live_text_updates';  
    contextData: 'patient_history_emr_data';  
  };  
  
  parallelProcessing: {  
    soapNoteGeneration: {  
      trigger: 'continuous';  
      template: 'practice_specific';  
      medicalShorthand: 'enabled';  
      emrFieldMapping: 'configured_per_practice';  
    };  
  
    billingCodeSuggestion: {  
      trigger: 'pre_analysis_and_continuous';  
      confidence: 'priority_ordering';  
      updates: 'dynamic_based_on_conversation';  
    };  
  
    orderSuggestion: {  
      trigger: 'symptom_mention_and_assessment';  
      priorAuth: 'real_time_checking';  
      formulary: 'insurance_verification';  
    };  
  };  
  
  outputCoordination: {  
    displayTiming: 'staggered_by_priority';  
    conflictHandling: 'highlight_differences';  
    doctorOverride: 'always_available';  
  };  
}
```

5. SOAP Note Generation & Templates

5.1 Template System Architecture

Template Selection Logic:


```
interface SOAPTemplateSystem {
  selectionCriteria: {
    visitType: 'new_patient' | 'follow_up' | 'urgent' | 'procedure';
    providerPreference: 'doctor_specific_templates';
    patientCondition: 'condition_specific_sections';
    specialty: 'internal_medicine' | 'cardiology' | 'endocrinology'; // etc.
  };

  templateComponents: {
    subjective: {
      chiefComplaint: 'patient_words';
      historyPresentIllness: 'chronological_narrative';
      reviewOfSystems: 'positive_negative_findings';
      pastMedicalHistory: 'relevant_conditions';
      medications: 'current_with_adherence';
      allergies: 'verified_reactions';
      socialHistory: 'relevant_lifestyle_factors';
    };

    objective: {
      vitalSigns: 'current_measurements';
      physicalExam: 'system_based_findings';
      labResults: 'relevant_current_results';
      imaging: 'pertinent_findings';
    };

    assessment: {
      primaryDiagnosis: 'icd10_coded';
      differentialDiagnosis: 'considered_alternatives';
      comorbidities: 'relevant_conditions';
      riskStratification: 'calculated_scores';
    };

    plan: {
      medications: 'new_changes_monitoring';
      orders: 'labs_imaging_procedures';
      followUp: 'timing_and_purpose';
      patientEducation: 'key_counseling_points';
      monitoring: 'safety_parameters';
    };
  };
}
```

Medical Shorthand & EMR Integration:

typescript

```
interface MedicalShorthandSystem {
  predefinedLibrary: {
    commonAbbreviations: 'BP, HR, RR, O2 sat, etc.';
    symptomDescriptors: 'SOB, CP, HA, N/V, etc.';
    physicalExamFindings: 'RRR, CTAB, soft NT ND, etc.';
    planElements: 'RTC, F/U, PRN, etc.';
  };

  practiceCustomization: {
    doctorSpecific: 'personal_abbreviation_preferences';
    specialtySpecific: 'cardiology_vs_endocrinology_terms';
    facilitySpecific: 'institutional_standards';
  };

  emrFieldMapping: {
    challenge: 'most_complex_piece';
    requirements: [
      'Map SOAP sections to specific EMR fields',
      'Handle varying EMR data structures',
      'Maintain original formatting preferences',
      'Support multiple EMR systems simultaneously'
    ];
  };

  implementationStrategy: {
    emrConfiguration: 'per_practice_field_mapping';
    dataTransformation: 'ai_assisted_field_population';
    validationRules: 'emr_specific_requirements';
    fallbackMethod: 'structured_export_for_copy_paste';
  };
};
```

5.2 Real-time Note Generation

Live Note Population:

typescript


```
interface LiveNoteGeneration {
  realTimeUpdates: {
    frequency: 'every_30_seconds';
    sections: 'populate_as_discussed';
    confidence: 'highlight_uncertain_text';
    doctorReview: 'final_approval_required';
  };

  qualityAssurance: {
    medicalAccuracy: 'terminology_validation';
    completeness: 'required_section_checking';
    billing_compliance: 'documentation_level_verification';
    emrCompatibility: 'field_requirement_validation';
  };

  editingWorkflow: {
    liveEditing: 'doctor_can_modify_during_encounter';
    versionControl: 'track_ai_vs_doctor_changes';
    autoSave: 'every_60_seconds';
    conflictResolution: 'doctor_changes_always_win';
  };
}
```

6. Billing Code & Order Management

6.1 Live Billing Code Suggestions

Code Generation Pipeline:

```
typescript
```

```

interface BillingCodeSystem {
  triggers: {
    preAnalysis: 'based_on_scheduled_visit_type';
    liveTriggers: [
      'chief_complaint_mentioned',
      'physical_exam_performed',
      'procedures_discussed',
      'time_spent_documented'
    ];
  };

  codeTypes: {
    evaluation: 'CPT 99XXX codes';
    procedures: 'CPT procedure codes';
    diagnoses: 'ICD-10-CM codes';
    modifiers: 'appropriate_billing_modifiers';
  };

  displayLogic: {
    priorityOrdering: 'confidence_score_ranking';
    dynamicUpdates: 'change_as_encounter_evolves';
    conflicts: 'highlight_mutually_exclusive';
    documentation: 'link_to_supporting_note_sections';
  };

  validation: {
    medicalNecessity: 'diagnosis_procedure_alignment';
    insurance: 'coverage_verification';
    compliance: 'coding_guideline_adherence';
    bundling: 'detect_inappropriate_unbundling';
  };
}

```

Code Approval Workflow:

typescript

```
interface CodeApprovalWorkflow {
  doctorReview: {
    required: true;
    options: ['accept', 'modify', 'reject', 'add_custom'];
    bulkActions: ['accept_all', 'review_individually'];
  };

  documentation: {
    openQuestion: 'ensure_sufficient_documentation_for_selected_codes';
    validation: 'warn_if_documentation_insufficient';
    suggestions: 'recommend_additional_documentation';
  };

  finalSubmission: {
    timing: 'with_soap_note_signing';
    validation: 'final_compliance_check';
    submission: 'to_billing_portal_for_claim_creation';
  };
}
```

6.2 Order Management System

Live Order Suggestions:

typescript

```
interface OrderManagementSystem {  
  orderTypes: {  
    laboratory: 'based_on_symptoms_and_conditions';  
    imaging: 'appropriate_study_recommendations';  
    prescriptions: 'medication_therapy_suggestions';  
    referrals: 'specialist_consultation_needs';  
    procedures: 'in_office_or_outpatient_procedures';  
  };  
  
  priorAuthDetection: {  
    realTimeChecking: 'insurance_database_lookup';  
    flagging: 'highlight_requires_prior_auth';  
    workflow: 'automated_prior_auth_initiation';  
    alternatives: 'suggest_covered_alternatives';  
  };  
  
  drugInteractionChecking: {  
    currentMedications: 'real_time_interaction_screening';  
    allergies: 'contraindication_alerts';  
    kidneyFunction: 'renal_dosing_adjustments';  
    drugFormulary: 'insurance_preferred_medications';  
  };  
}
```

7. Post-Visit Care Note Generation

7.1 Patient Care Note System

Multi-Factor Note Generation:

typescript

```
interface PostVisitCareNotes {
  generationTriggers: {
    all: ['condition', 'visit_type', 'patient_demographics'];
    timing: 'immediate_post_live_scribe_completion';
    aiModel: 'patient_education_specialist';
  };

  contentPersonalization: {
    byCondition: {
      diabetes: 'glucose_monitoring_diet_exercise_guidelines';
      hypertension: 'blood_pressure_management_lifestyle_modifications';
      heartFailure: 'fluid_restriction_weight_monitoring_medication_adherence';
      copd: 'breathing_techniques_medication_usage_activity_modification';
    };

    byVisitType: {
      newPatient: 'comprehensive_introduction_to_care_team_and_portal';
      followUp: 'progress_review_and_next_steps';
      urgent: 'immediate_care_instructions_and_warning_signs';
      procedure: 'post_procedure_care_instructions_and_expectations';
    };

    byDemographics: {
      pediatric: 'age_appropriate_language_parent_caregiver_instructions';
      geriatric: 'simplified_instructions_large_font_caregiver_copy';
      language: 'patient_preferred_language_cultural_considerations';
      literacy: 'health_literacy_level_appropriate_complexity';
    };
  };

  languageSupport: {
    multiLanguage: true;
    supportedLanguages: ['english', 'spanish', 'chinese', 'vietnamese', 'arabic', 'russian'];
    translationMethod: 'ai_medical_translation_with_human_review';
    culturalAdaptation: 'region_specific_medical_practices';
  };

  deliveryMethods: {
    patientPortal: 'primary_delivery_with_read_confirmation';
    email: 'secondary_option_with_pdf_attachment';
    sms: 'summary_with_portal_link';
    print: 'in_office_printing_for_patients_without_digital_access';
  };
}
```

```
};  
}
```

Content Structure & Quality:

typescript

```
interface CareNoteContent {  
  simpleLanguage: {  
    readingLevel: 'grade_6_maximum';  
    sentenceLength: 'average_15_words_or_fewer';  
    medicalTerms: 'explained_in_parentheses_or_avoided';  
    actionItems: 'clear_numbered_lists';  
  };  
  
  standardSections: {  
    visitSummary: 'what_we_discussed_today_in_simple_terms';  
    medications: 'what_changed_why_how_to_take_properly';  
    nextSteps: 'what_you_need_to_do_before_next_visit';  
    warningSigns: 'when_to_call_office_or_seek_emergency_care';  
    followUp: 'when_where_why_for_next_appointment';  
    resources: 'helpful_websites_support_groups_educational_materials';  
  };  
  
  qualityAssurance: {  
    accuracyCheck: 'match_against_soap_note_and_encounter_transcript';  
    completenessCheck: 'all_patient_questions_addressed';  
    clarityCheck: 'health_literacy_validation';  
    doctorReview: 'optional_review_before_sending_to_patient';  
  };  
}
```

8. Split-Screen Mode & Voice Commands

8.1 Test Result Display System

Voice Command Processing:

typescript

```
interface VoiceCommandSystem {
  prefetchedData: {
    labResults: 'all_historical_lab_values';
    imagingReports: 'radiology_and_cardiology_studies';
    consultNotes: 'specialist_reports_and_recommendations';
    trendData: 'longitudinal_value_tracking';
  };

  semanticMatching: {
    labCommands: [
      'Show latest CBC',
      'Display A1C trend',
      'Pull up liver function',
      'Show lipid panel'
    ];

    imagingCommands: [
      'Display last echo',
      'Show chest X-ray',
      'Pull up CT scan'
    ];

    trendCommands: [
      'Show blood pressure trend',
      'Display weight over time',
      'Chart glucose levels'
    ];
  };

  displayPriority: {
    exactMatch: 'priority_1';
    semanticMatch: 'priority_2_with_confidence_score';
    dateRelevance: 'recent_results_prioritized';
    abnormalValues: 'flagged_results_highlighted';
  };
}
```

Split-Screen Layout:

typescript

```
interface SplitScreenLayout {
  activation: {
    buttonClick: 'toggle_button_in_scribe_section';
    voiceCommand: 'say_show_split_screen';
  };

  layoutManagement: {
    desktop: '60_40_split_scribe_left_results_right';
    laptop: '50_50_split_with_responsive_text_sizing';
    tablet: 'overlay_mode_with_quick_toggle';
  };

  resultSelection: {
    multipleMatches: 'present_options_for_doctor_selection';
    quickSelection: 'number_shortcuts_1_2_3';
    persistence: 'remember_frequently_accessed_results';
  };
}
```

8. Real-time Synchronization & State Management

8.1 Cross-Portal Integration

Patient-Level Atomic Operations:

```
typescript
```



```

interface PatientFileManagement {
  atomicOperations: {
    concept: 'check_in_check_out_at_patient_file_level';
    lockingStrategy: 'patient_record_level_locking';
    conflictPrevention: 'single_active_encounter_per_patient';
  };

  encounterStateMachine: {
    states: [
      'scheduled',
      'checked_in',
      'roomed',
      'ready_for_doctor',
      'encounter_in_progress', // Doctor has exclusive access
      'encounter_completed',
      'notes_signed',
      'billing_submitted'
    ];

    exclusiveAccess: {
      during: 'encounter_in_progress';
      owner: 'attending_doctor';
      exceptions: 'ma_assistance_requests_only';
    };
  };

  communicationRules: {
    duringEncounter: {
      doctorToMA: 'can_send_assistance_requests';
      maToDoctor: 'cannot_interrupt_encounter';
      otherStaff: 'view_only_status';
    };

    postEncounter: {
      realTimeUpdates: 'immediate_status_broadcast_to_all_portals';
      dataSync: 'encounter_data_propagation';
      workflowTriggers: 'billing_patient_notification_scheduling';
    };
  };
}

```

Real-time Status Broadcasting:

typescript

```
interface StatusBroadcastSystem {  
  encounterEvents: {  
    encounterStart: {  
      broadcast: 'patient_status_in_progress_to_all_portals';  
      recipients: ['ma_dashboard', 'front_office', 'billing_portal'];  
      data: ['patient_id', 'doctor_id', 'start_time', 'estimated_duration'];  
    };  
  
    maAssistanceRequest: {  
      broadcast: 'urgent_notification_to_ma_dashboard';  
      recipients: ['ma_dashboard'];  
      data: ['patient_id', 'room_number', 'assistance_type', 'doctor_notes'];  
      priority: 'high';  
    };  
  
    encounterComplete: {  
      broadcast: 'status_update_and_workflow_triggers';  
      recipients: ['all_portals'];  
      data: ['patient_id', 'completion_time', 'next_steps', 'billing_ready'];  
      triggers: ['billing_workflow', 'patient_care_note_generation'];  
    };  
  };  
  
  realTimeSync: {  
    technology: 'websocket_connections_with_fallback_polling';  
    frequency: 'immediate_for_critical_updates';  
    reliability: 'guaranteed_delivery_with_retry_logic';  
    scope: 'practice_wide_with_patient_filtering';  
  };  
}
```

8.2 EMR Synchronization

Post-Encounter Sync Strategy:

typescript

```

interface EMRSyncSystem {
  syncTiming: {
    primary: 'post_encounter_completion';
    fallback: 'manual_sync_button_for_technical_issues';
    monitoring: 'sync_status_indicators_and_alerts';
  };

  dataIntegrity: {
    validation: 'pre_sync_data_validation_checks';
    rollback: 'ability_to_rollback_failed_syncs';
    manualOverride: 'doctor_can_force_sync_or_export_data';
    auditTrail: 'complete_sync_history_and_error_logs';
  };

  syncComponents: {
    soapNotes: 'structured_data_to_emr_note_fields';
    billingCodes: 'direct_integration_with_emr_billing_module';
    orders: 'cpoe_integration_or_structured_order_export';
    vitals: 'discrete_data_field_population';
    medications: 'medication_reconciliation_updates';
  };

  failureHandling: {
    networkIssues: 'queue_for_retry_with_exponential_backoff';
    emrDowntime: 'local_storage_with_sync_when_available';
    dataConflicts: 'alert_doctor_with_merge_options';
    criticalFailures: 'emergency_data_export_and_manual_entry_workflow';
  };
}

```

9. Performance & Technical Requirements

9.1 Performance Standards

Scalability Requirements:

typescript

```
interface ScalabilityRequirements {
  concurrentUsers: {
    minimum: '500_providers_across_multiple_practices';
    infrastructure: 'gcp_auto_scaling_architecture';
    resourceAllocation: 'patient_care_takes_priority';
    loadBalancing: 'intelligent_traffic_distribution';
  };

  hardwareRequirements: {
    deviceType: 'modern_laptop_minimum';
    specifications: {
      ram: '8gb_minimum_16gb_recommended';
      processor: 'intel_i5_or_amd_equivalent_or_better';
      storage: '256gb_ssd_minimum';
      network: '25mbps_minimum_50mbps_recommended';
      browser: 'chrome_firefox_safari_latest_versions';
    };
  };

  gcpInfrastructure: {
    services: [
      'cloud_run_for_containerized_applications',
      'vertex_ai_for_ai_processing',
      'cloud_sql_for_structured_data',
      'firestore_for_real_time_synchronization',
      'cloud_storage_for_file_handling',
      'cloud_cdn_for_global_performance'
    ];
  };

  autoScaling: {
    triggers: 'cpu_memory_usage_and_concurrent_user_metrics';
    scaling: 'horizontal_pod_autoscaling';
    resourceLimits: 'patient_care_priority_no_artificial_limits';
  };
};

performanceMetrics: {
  pageLoad: '<2_seconds_95th_percentile';
  aiProcessing: '<5_seconds_for_all_agents';
  emrSync: '<10_seconds_post_encounter';
  realTimeUpdates: '<1_second_cross_portal_propagation';
};
```

```
};  
}
```

10. Quality Assurance & Clinical Safety

10.1 Clinical Decision Support & Safety

Human-in-the-Loop Safety Systems:

typescript

```

interface ClinicalSafetySystem {
  humanInLoop: {
    principle: 'all_ai_suggestions_require_doctor_review_and_approval';
    implementation: 'explicit_accept_reject_modify_options';
    override: 'doctor_can_always_override_ai_recommendations';
    documentation: 'log_all_ai_suggestions_and_doctor_decisions';
  };

  drugInteractionAlerts: {
    display: 'red_lettering_and_modal_card';
    severity: ['minor', 'moderate', 'major', 'contraindicated'];
    sources: 'lexicomp_micromedex_fdb_databases';
    workflow: 'must_acknowledge_before_proceeding';
    alternatives: 'suggest_safer_medication_options';
  };

  allergyChecking: {
    crossReference: 'all_medications_against_known_allergies';
    severity: 'rank_by_allergy_severity_and_reaction_type';
    display: 'prominent_warning_with_historical_reaction_details';
    documentation: 'require_allergy_verification_and_update';
  };

  criticalValueAlerts: {
    labResults: 'immediate_notification_for_life_threatening_values';
    vitalSigns: 'real_time_alerts_for_dangerous_vital_signs';
    display: 'modal_card_with_red_highlighting';
    escalation: 'require_immediate_doctor_action_or_acknowledgment';
    followUp: 'automatic_tracking_of_critical_value_response';
  };

  clinicalGuidelines: {
    integration: 'uptodate_acp_guidelines_cdc_recommendations';
    alerts: 'guideline_based_care_reminders';
    evidence: 'link_to_supporting_clinical_evidence';
    customization: 'practice_specific_protocol_integration';
  };
}

```

11. Integration & Interoperability

11.1 External System Integration

ePrescribing Workflow:

typescript

```
interface ePrescribingIntegration {  
  workflow: {  
    step1: 'ai_suggests_medications_during_live_scribe';  
    step2: 'doctor_reviews_and_approves_in_sarathi';  
    step3: 'medication_data_syncs_to_emr';  
    step4: 'emr_triggers_eprescribing_system';  
    philosophy: 'not_reinventing_the_wheel';  
  };  
  
  integrationPoints: {  
    emrIntegration: 'leverage_existing_emr_eprescribing_capabilities';  
    dataTransfer: 'structured_medication_data_to_emr';  
    monitoring: 'track_prescription_status_through_emr';  
    fallback: 'paper_prescription_generation_if_eprescribing_fails';  
  };  
  
  prescriptionData: {  
    medication: 'generic_and_brand_names';  
    dosage: 'strength_frequency_duration';  
    quantity: 'pills_or_volume_with_refills';  
    instructions: 'patient_specific_directions';  
    priorAuth: 'insurance_authorization_requirements';  
    formulary: 'insurance_preferred_alternatives';  
  };  
}
```

Laboratory Integration:

typescript

```
interface LaboratoryIntegration {  
  orderPlacement: {  
    method: 'api_based_lab_system_integration';  
    standards: 'hl7_fhir_or_lab_specific_apis';  
    workflow: 'direct_order_transmission_from_sarthi';  
    fallback: 'structured_export_for_manual_entry';  
  };  
  
  resultRetrieval: {  
    method: 'api_based_automated_result_polling';  
    frequency: 'real_time_push_or_frequent_polling';  
    processing: 'ai_analysis_and_technician_review';  
    integration: 'seamless_import_to_patient_chart';  
  };  
  
  labSystems: {  
    commercial: 'quest_labcorp_others_via_api';  
    hospital: 'epic_cerner_lab_modules';  
    independent: 'custom_api_integrations';  
    legacy: 'hl7_interface_engines';  
  };  
}
```

Insurance Verification:

typescript


```
interface InsuranceIntegration {  
  realTimeVerification: {  
    method: 'api_based_eligibility_checking';  
    providers: 'availability_change_healthcare_others';  
    timing: 'pre_visit_analysis_and_during_encounter';  
    coverage: 'verify_specific_procedures_and_medications';  
  };  
  
  priorAuthorization: {  
    detection: 'automatic_identification_of_prior_auth_requirements';  
    workflow: 'automated_prior_auth_initiation_where_possible';  
    tracking: 'status_monitoring_and_approval_notifications';  
    alternatives: 'suggest_covered_alternatives_if_denied';  
  };  
  
  copayCalculation: {  
    realTime: 'calculate_patient_responsibility';  
    integration: 'billing_system_coordination';  
    communication: 'transparent_cost_disclosure_to_patients';  
  };  
}
```

11.2 Data Synchronization & Backup

Manual Sync Capabilities:

typescript

```
interface ManualSyncSystem {
  triggers: {
    syncFailure: 'automatic_fallback_to_manual_options';
    userInitiated: 'doctor_can_manually_trigger_sync';
    emergencyMode: 'critical_patient_data_export';
  };

  manualOptions: {
    dataExport: {
      format: 'structured_ccda_or_hl7_export';
      content: 'complete_encounter_documentation';
      delivery: 'secure_file_transfer_or_direct_import';
    };

    printOptions: {
      encounterSummary: 'formatted_visit_summary_for_scanning';
      prescriptions: 'paper_prescriptions_with_barcodes';
      labOrders: 'requisition_forms_with_patient_identifiers';
    };

    directEntry: {
      emrAccess: 'instructions_for_manual_emr_entry';
      dataValidation: 'checklists_to_ensure_complete_transfer';
      auditTrail: 'log_manual_entry_completion';
    };
  };

  recoveryProcedures: {
    dataIntegrity: 'verify_all_critical_data_transferred';
    followUp: 'schedule_tech_support_for_sync_repair';
    patientCare: 'ensure_no_interruption_to_patient_care';
  };
}
```

This comprehensive specification should provide your development team with clear, unambiguous requirements for implementing the sophisticated Doctor Dashboard with real-time AI assistance and EMR integration.

9. Performance & Technical Requirements

9.1 Performance Standards

Response Time Requirements:

``typescript

```
interface PerformanceStandards {  
  pageLoad: '<2_seconds';  
  aiAnalysisGeneration: '<30_seconds';  
  liveScribeLatency: '<3_seconds';  
  voiceCommandResponse: '<2_seconds';  
  emrSync: '<5_seconds';  
  billingCodeSuggestions: '<5_seconds';  
  
  concurrentUsers: {  
    perPractice: 'up_to_20_providers_simultaneously';  
    systemWide: 'unlimited_with_gcp_auto_scaling';  
  };  
  
  offlineMode: {  
    cacheSize: 'up_to_50mb_encrypted_patient_data';  
    syncTime: '<30_seconds_on_reconnection';  
    conflictResolution: '<10_seconds';  
  };  
}
```

9.2 Security & Compliance

HIPAA Compliance:

typescript

```
interface SecurityRequirements {  
  dataEncryption: {  
    inTransit: 'TLS_1_3_minimum';  
    atRest: 'AES_256_encryption';  
    localStorage: 'encrypted_with_user_session_key';  
  };  
  
  auditLogging: {  
    patientAccess: 'every_chart_open_and_close';  
    dataModification: 'all_note_edits_and_code_changes';  
    aiInteractions: 'all_ai_suggestions_and_doctor_responses';  
    emrSync: 'all_data_synchronization_events';  
  };  
  
  accessControl: {  
    sessionTimeout: '30_minutes_inactivity';  
    deviceRegistration: 'optional_for_high_security_practices';  
    multiFactorAuth: 'configurable_requirement';  
  };  
}
```

10. Testing & Quality Assurance

10.1 Testing Requirements

Functional Testing:

typescript

```

interface TestingRequirements {
  unitTests: [
    'AI agent coordination logic',
    'Voice command semantic matching',
    'Billing code validation',
    'EMR field mapping',
    'Template population accuracy'
  ];

  integrationTests: [
    'Live scribe with multiple AI agents',
    'EMR synchronization with conflict resolution',
    'Cross-portal state management',
    'Offline mode and sync recovery'
  ];

  performanceTests: [
    'Concurrent user load testing',
    'AI processing under heavy load',
    'Network degradation scenarios',
    'Large patient data handling'
  ];

  clinicalAccuracyTests: [
    'Medical terminology recognition',
    'SOAP note clinical validity',
    'Billing code appropriateness',
    'Drug interaction detection'
  ];
}

```

10.2 Acceptance Criteria

Core Functionality:

- **Dashboard must load within 2 seconds** with real-time EMR sync status
- **Pre-analysis must complete within 30 seconds** of patient data changes
- **Live scribe must maintain <3 second latency** for transcription and AI processing
- **Voice commands must respond within 2 seconds** with accurate semantic matching
- **All patient state changes must sync across portals within 30 seconds**
- **Offline mode must cache and sync all data** without loss when connectivity restored

- **All AI suggestions must be reviewable and modifiable** by the doctor
- **EMR integration must support bidirectional sync** with conflict resolution
- **Split-screen mode must work seamlessly** on laptop and desktop devices
- **Billing codes must update dynamically** during the encounter
- **SOAP notes must populate in real-time** using practice-specific templates
- **All interactions must be HIPAA compliant** with complete audit trails

This comprehensive specification should provide your development team with clear, unambiguous requirements for implementing the sophisticated Doctor Dashboard with real-time AI assistance and EMR integration.